
Web Privacy - Lab 1

Stateful Web Tracking

Student: Zakariya Sabri
Topic: HTTP headers, cookies and web tracking
Date: September 2026

This report summarizes the experiments and the requested observations.

1. HTTP headers

The Flask server returned an HTML page and added the custom response header `X-Lab-Message`. The Network panel was used to compare the headers sent by the browser with those printed by Flask.

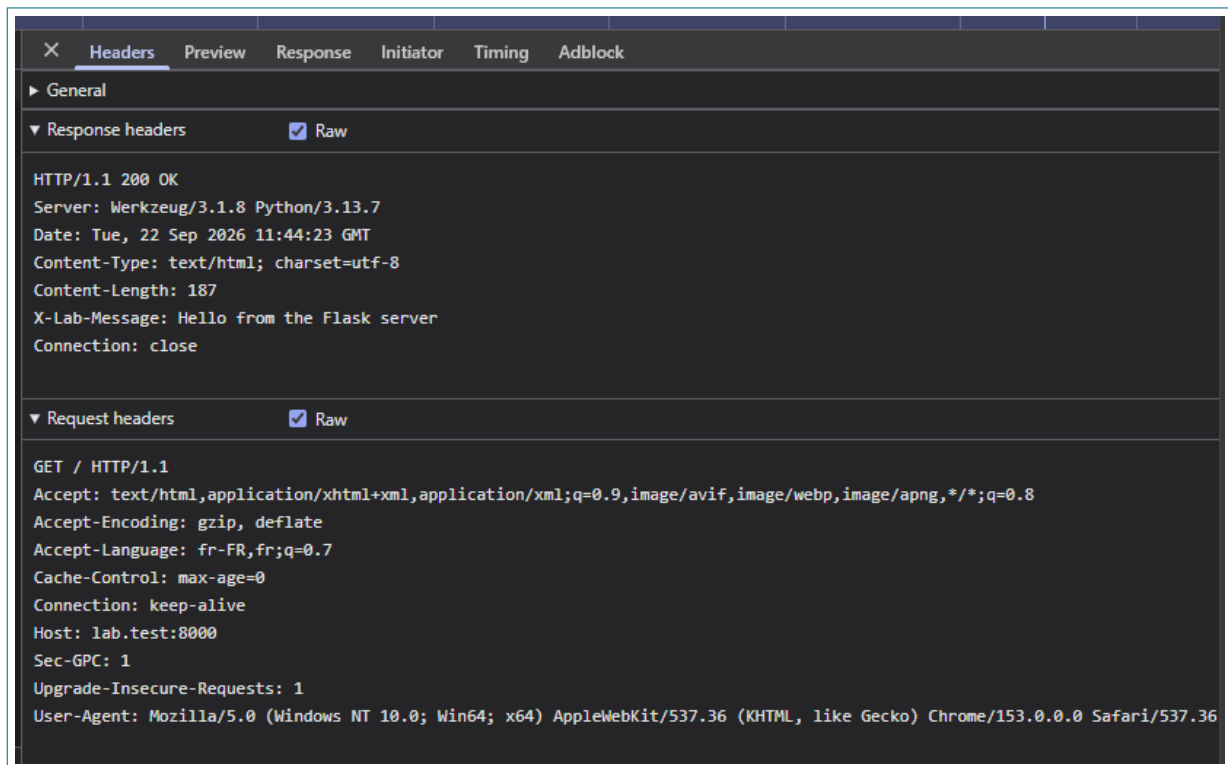


Figure 1: Request and response headers in the browser.

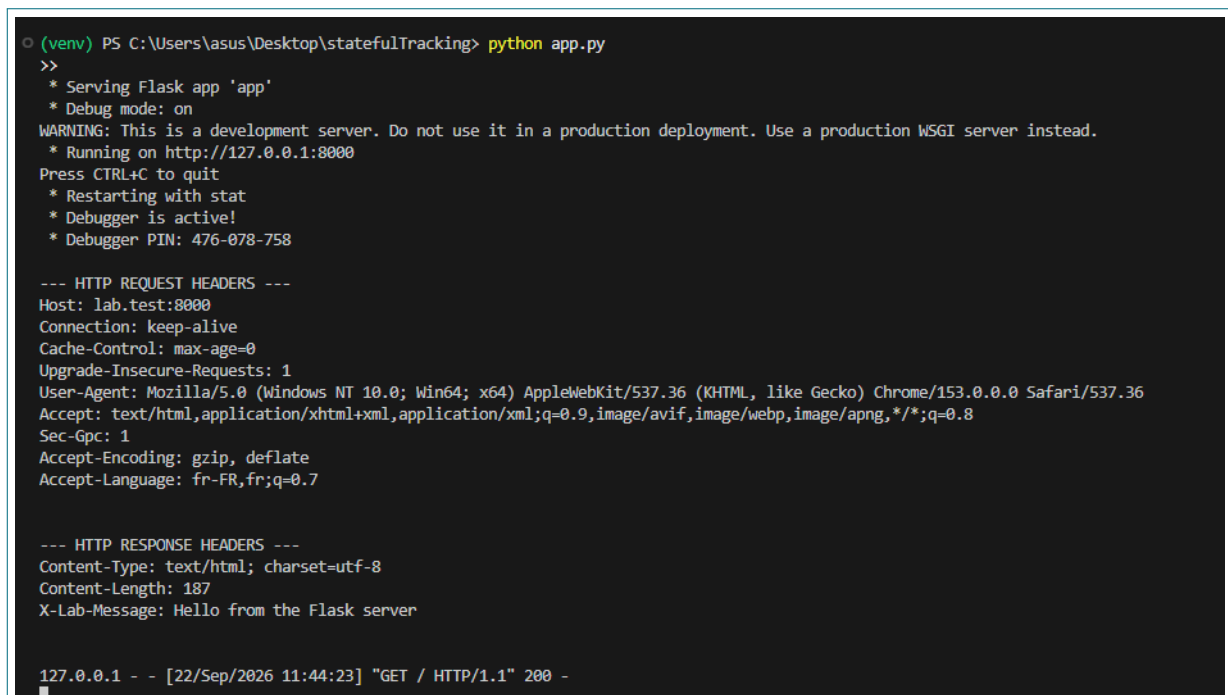


Figure 2: Headers observed from the Flask server.

2. Cookie creation and attributes

The server generated a random `aid` identifier and returned it in a `Set-Cookie` response header. On later requests, the browser returned it using the `Cookie` request header.

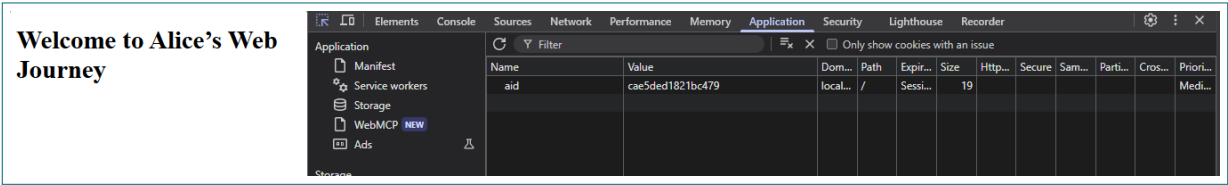


Figure 3: The browser stores the newly created `aid` cookie.

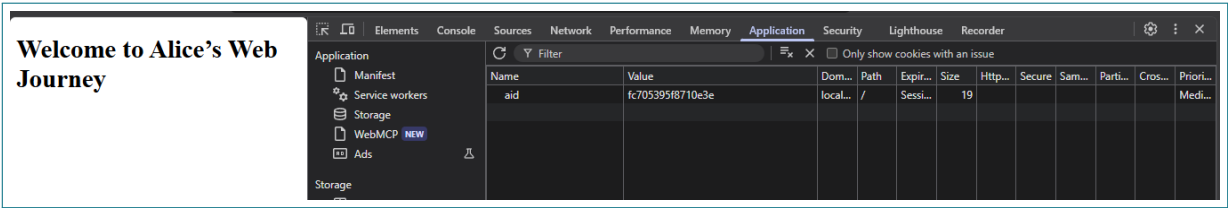


Figure 4: The previous cookie is deleted before a new attribute test.

Cookie attributes tested

- **Max-Age**: keeps the cookie after the browser restarts.
- **HttpOnly**: prevents JavaScript from reading the cookie.
- **Secure**: restricts transmission to HTTPS.
- **Path**: selects the routes to which the cookie is sent.
- **SameSite**: controls transmission in cross-site requests.

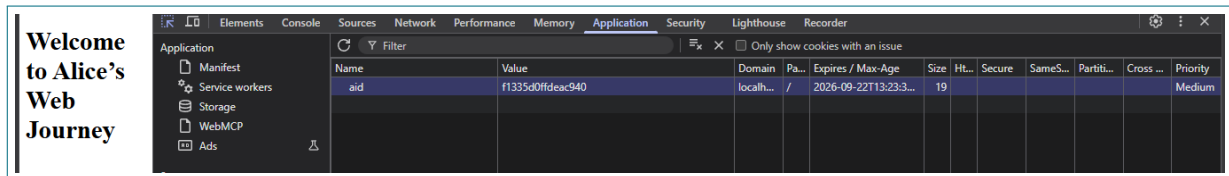


Figure 5: Persistent cookie configured with **Max-Age**.

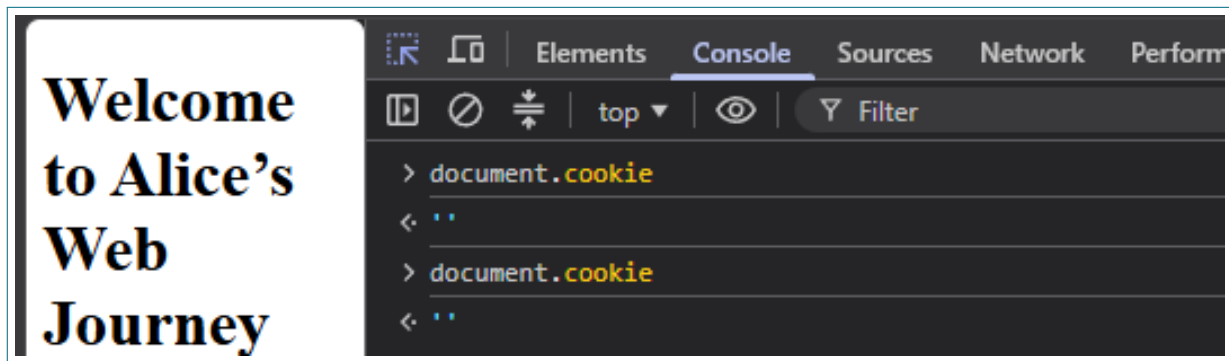


Figure 6: The **HttpOnly** attribute is enabled.

Question: Is the original cookie still present after closing and restarting the browser? Does it have the same identifier?

Answer. Without **Max-Age** or **Expires**, it is a session cookie and is normally removed when the browser session ends. A persistent cookie remains available until its expiration and keeps the same identifier. Browser session restoration can preserve a session cookie.

3. Challenge 1 - Third-party tracking

Two independent publishers embedded the same tracker in an `iframe`. The tracker received the publisher and page in query parameters, created its own `aid` cookie, and recorded visits chronologically.

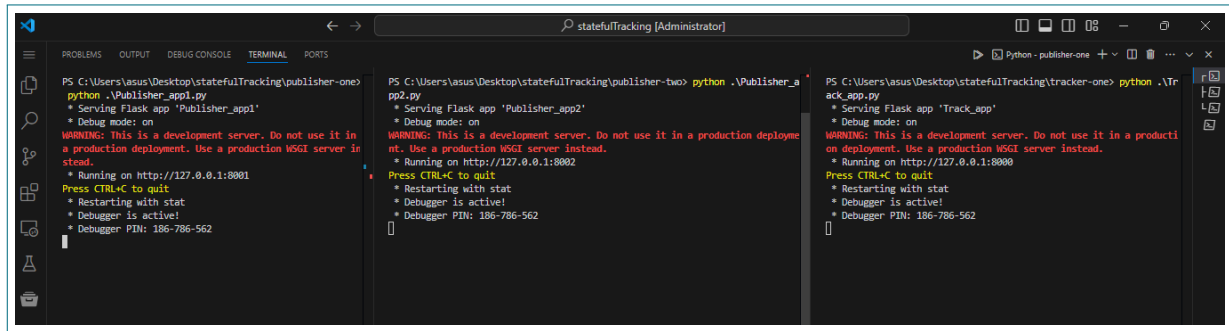


Figure 7: Publisher 1 and Publisher 2 run independently.

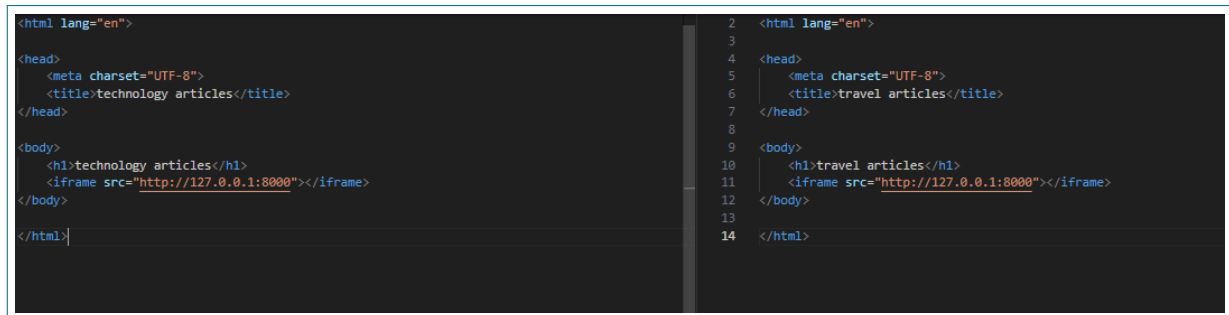


Figure 8: A publisher embeds content from `tracker-one.test`.

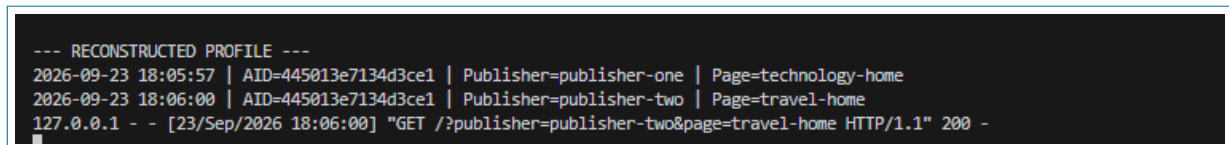
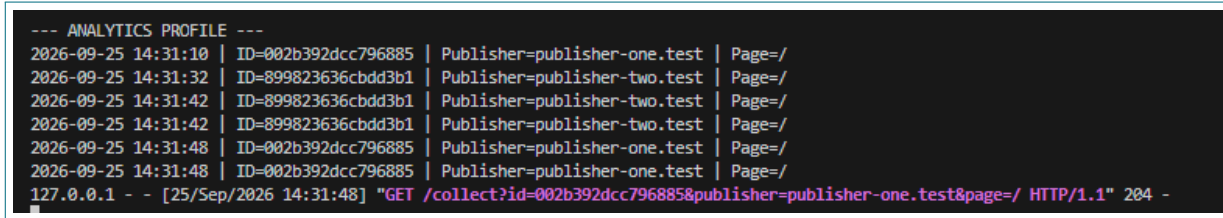


Figure 9: Chronological profile reconstructed by the tracker.

The tracker cookie belongs to `tracker-one.test`, not to either publisher. Modern browsers may omit or partition this cookie in a third-party iframe. In particular, an unspecified `SameSite` value is generally treated as `Lax`; the iframe request is still sent, but the cookie may be excluded.

4. Challenge 2 - First-party analytics

Both publishers loaded the same `analytics.js` file. Although the file came from `analytics.test`, it executed in the context of the publisher. It created an `analytics_id` cookie and explicitly sent the identifier, publisher and page to the analytics server.



```
--- ANALYTICS PROFILE ---
2026-09-25 14:31:10 | ID=002b392dcc796885 | Publisher=publisher-one.test | Page=/
2026-09-25 14:31:32 | ID=899823636cbdd3b1 | Publisher=publisher-two.test | Page=/
2026-09-25 14:31:42 | ID=899823636cbdd3b1 | Publisher=publisher-two.test | Page=/
2026-09-25 14:31:42 | ID=899823636cbdd3b1 | Publisher=publisher-two.test | Page=/
2026-09-25 14:31:48 | ID=002b392dcc796885 | Publisher=publisher-one.test | Page=/
2026-09-25 14:31:48 | ID=002b392dcc796885 | Publisher=publisher-one.test | Page=/
127.0.0.1 - - [25/Sep/2026 14:31:48] "GET /collect?id=002b392dcc796885&publisher=publisher-one.test&page=/ HTTP/1.1" 204 -
```

Figure 10: Profiles collected by the first-party analytics service.

Question: Are the two publishers assigned the same identifier or different identifiers?

Answer. They are assigned different identifiers.

Question: Why does this behaviour occur?

Answer. The script executes independently in each publisher's context. Cookies are isolated by domain, so a cookie created on `publisher-one.test` cannot be read on `publisher-two.test`.

Question: Does the analytics server automatically receive the first-party cookies?

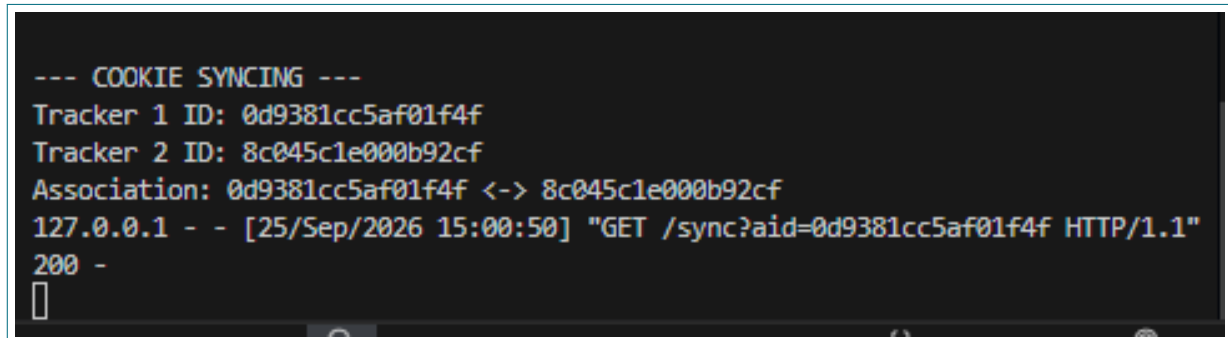
Answer. No. Those cookies belong to the publishers. JavaScript must read the identifier and explicitly send it to the analytics server, for example through the `/collect` request.

Question: Does the analytics service create one combined profile or multiple separate profiles?

Answer. It creates multiple separate profiles because each publisher has a different first-party identifier. The service cannot combine them automatically without another linking mechanism.

5. Optional challenge - Cookie syncing

Tracker 1 owned `aid`; Tracker 2 owned `bid`. Domain isolation prevented either tracker from directly reading the other's cookie. Tracker 1 therefore sent its identifier in a redirect URL. Tracker 2 read its own cookie and stored the association `aid ↔ bid`.

A terminal window with a black background and white text. The text shows a sequence of log messages for a cookie syncing process. It starts with a separator line, followed by the IDs of two trackers, then an association line, and finally an HTTP log entry for a GET request to a sync endpoint.

```
--- COOKIE SYNCING ---  
Tracker 1 ID: 0d9381cc5af01f4f  
Tracker 2 ID: 8c045c1e000b92cf  
Association: 0d9381cc5af01f4f <-> 8c045c1e000b92cf  
127.0.0.1 - - [25/Sep/2026 15:00:50] "GET /sync?aid=0d9381cc5af01f4f HTTP/1.1"  
200 -  
█
```

Figure 11: Server-side association between the two tracker identifiers.

Answer. Cookie syncing allows two isolated tracker identifiers to be linked to the same browser without one tracker directly reading the other domain's cookie.

Conclusion

The lab demonstrated that browser recognition depends on cookies and HTTP headers. Third-party cookies can support cross-site profiles but are restricted by modern privacy protections. First-party analytics avoids third-party cookie storage, yet produces separate identifiers per publisher. Cookie syncing can link identifiers across tracker domains.